

Title	Severity	Evidence	Actual	Expected	Business Impact
Account create request properties are not required	P1	"handler.createAccountRequest" defines "document_number", but the schema has no "required" list.	Request body is required, but its "document_number" is optional.	document_number should be required if needed to create an account.	Invalid accounts may be created.
"document_number" has no validation rule	P1	"document_number" is only declared as a string with an example. There is no "minLength", "maxLength", "pattern", or other format constraint. Hearsay say "10 to 14 digits".	Any string is schema-valid, including an empty string or a string containing non-digit characters.	Contract should define the valid document-number shape.	Invalid account identities can enter the system and impact reconciliation.
Duplicate "document_number" behavior is undefined	P1	"POST /accounts" documents only 201, 400, and 405. No uniqueness rule is defined.	Contract does not say whether duplicate documents are allowed, rejected, or mapped to an existing account.	If "document_number" identifies a unique cardholder account, duplicate creation should have an explicit outcome.	Ambiguous duplicate handling can create multiple accounts with one document_number and attach transactions to incorrect account.
POST /accounts error messages are not mapped to HTTP status codes	P1	"POST /accounts" documents 400 and 405 with error message "account not found".	"POST /accounts" documents 400 and 405 with error message "account not found".	The condition and status should have a deterministic mapping.	Consumers may handle failures incorrectly.
GET /accounts/{accountId} error messages are not mapped to HTTP status codes	P1	"GET /accounts/{accountId}" documents 400, 404 and 405 with error message "account not found".	"GET /accounts/{accountId}" documents 400, 404 and 405 with error message "account not found".	The condition and status should have a deterministic mapping.	Consumers may handle failures incorrectly.
Transaction request properties are not required.	P1	"handler.createTransactionRequest" defines "account_id", "amount", and "operation_type_id", but the schema has no "required" list.	Request body is required, but its individual properties are optional.	Contract should identify which fields are mandatory for a valid create request.	Malformed transactions may reach business logic.
Non existent account behavior is not defined	P1	Contract documents 400 and 422 but does not attach either status to a specific business condition. The "errorResponse" example says "account not found".	The example suggests an "account not found error" exists, but the contract does not say whether it is 400 or 422.	The condition and status should have a deterministic mapping.	A transaction may be accepted against an invalid account.
"amount" has no decimal precision, range, or sign constraint.	P0	The request defines "amount" as only "number". The endpoint description says the caller sends a positive amount, but the schema itself does not enforce decimal precision, positivity, maximum value, or minimum value.	Zero, negative, and extremely large values are all schema-valid.	Contract should define the amount range and decimal precision to test it.	Incorrect monetary amount can corrupt balances and financial records.
Currency and API amount unit are unspecified.	P0	The request defines "amount" as only "number". There is no currency field or unit declaration anywhere in the supplied contract.	Backend engineer says "Amounts go in as rupees, we store them in the smallest unit internally.", but neither point appears in the contract.	API should make the amount representation unambiguous.	Caller and provider can interpret the same numeric amount differently, causing financial loss.
"operation_type_id" accepts any integer	P0	"operation_type_id" is declared only as an integer. The transaction description lists four operation IDs, but contract don't define any constraint.	Schema allows any integer.	Only supported operation types should be valid.	An unknown operation type can reach business logic changing money incorrectly.
Idempotency is absent from the contract	P0	No idempotency header is documented.	Backend engineer says "There's an idempotency header on transactions so retries don't double-charge. Send the same key, you get the original response back.", but is not represented in the contract.	Idempotency header must be documented.	Retries can lead to duplicate financial transactions.
Idempotency key reuse with a different body is undefined	P0	No idempotency header is documented.	No behavior is specified.	An idempotency key must not become associated with multiple financial operations.	One idempotency key could become associated with two different financial operations.
Operation-specific mapping is not contractual	P0	The contract says "Operation type IDs: 1=Normal Purchase, 2=Purchase with installments, 3=Withdrawal, 4=Credit Voucher. Send a positive amount. The server applies the correct sign based on operation type.". The response example contains "amount: -100.5" and "type: debit".	"amount" sign is not defined for each ID. The contract gives one negative example but does not define the complete mapping. Backend engineer says "Purchases and withdrawals go negative, credit vouchers go positive."	The mapping between each operation and its financial sign/type should be explicit.	An incorrect sign can lead to money integrity failure. E. g. sign/type inversion can debit instead of credit and vice versa.
"event_date" is a string without "date-time" format	P2	"event_date" is declared as "string" with an ISO-looking example.	Any string is schema-valid.	Event timestamp format should be contractual.	Consumers may parse timestamp incorrectly.
Response "type" is an unrestricted string	P1	The response defines "type" as string, example "debit".	Any string is schema-valid.	Transaction types should be constrained if consumers depend on it.	A consumer could receive an unexpected transaction type without violating the schema.
Audit-write guarantee is not represented	P0	Contract contains no response field, or header representing audit creation.	The audit-row statement exists only as Backend engineer hearsay.	If every write must produce an audit record, there needs to be a defined contract or a non-API verification mechanism.	A transaction and audit trail can diverge, breaking reconciliation.

Audit/business-write atomicity cannot be verified	P0	Contract doesn't define how database transactions are handled.	Backend engineer says "Every write also writes an audit row, in the same database transaction, so they can't diverge", but this is not present in contract.	Provider should either clearly guarantee that the operation happens completely or not at all in documentation, or provide a way for us to verify this through integration testing.	A transaction and audit trail can diverge, breaking reconciliation.
Request ID response header is not contractual	P1	Contract defines response bodies but no response headers.	No request ID header is documented.	Request ID header should be defined.	Lack of request ID slows financial incident investigation.
Response properties are not required	P1	Neither "accountResponse" nor "transactionResponse" declares a "required" list.	The schema allows a successful response with any or all properties absent.	Consumers should know which fields are guaranteed in a successful response.	For a transaction response, missing "transaction_id", "amount", "account_id", or operation information can make the result unusable for reconciliation.
Unknown-field handling is unspecified	P2	The contract is silent.	The contract is silent.	The provider should document whether unknown fields are ignored or rejected.	Consumer compatibility and validation behavior can vary.
Authentication/Authorization contract requirement is absent	P0	Contract does not define an authentication or authorization requirement for the Transactions Service.	No authentication/authorization requirement is documented.	Authentication and authorization requirements defined in the contract.	Unauthorized financial transactions may occur.